



Project – Airline Reservation System

Object Oriented Programming

(CSEG1044)

Phase 4: UI and Integration

Members:-

Yash Jangir – 590014858

Parth Choudhary-590016913

Dushyant Singh-590012592

Soham Sharma-590017041

Anuj Pandey-590017258

1. Executive Summary and Phase Goal

Goal of the Phase: The primary goal of Phase 4 is to build the main Java Swing graphical user interface (GUI) screens and seamlessly connect them to the service and Data Access Object (DAO) layers. This ensures that the airline application functions end-to-end, translating user interactions (like booking a flight or assigning a seat) into secure database transactions while adhering to strict Object-Oriented design principles.

2. Architectural Adherence and System Flow

This phase strictly follows the clean architecture established in previous phases.

- **System Flow:** Interactions must move sequentially: User -> UI Dashboard -> Service Layer (Logic Handler) -> DAO Layer-> DBConnection -> MySQL Database.
- **Separation of Concerns:** A core design rule strictly enforced in this phase is that absolutely no SQL queries are written within the UI layer.
- **Event Handling:** All UI button actions (e.g., actionPerformed events) are restricted to capturing input, performing visible UI validation, and delegating the data to the appropriate service methods.

3. Detailed UI Screen Implementations

To facilitate the operational flow of the system, the following interactive Swing screens will be developed. A custom AnimatedButton utility utilizing javax.swing.Timer can be implemented to provide smooth animations on mouse hover, enhancing user interaction.

Main Dashboard (MainDashboard.java)

- Acts as the primary container utilizing a CardLayout to manage sub-panels, preventing the UI anti-pattern of opening multiple overlapping windows.
- Features a persistent sidebar for clear, simple navigation between all major application modules (Routes, Flights, Bookings).

Route Management Screen (RoutePanel.java)

- **Purpose:** Facilitates the first step of the operational flow: "Add Route" (mirroring "Add Company").
- **Fields:** Captures Origin and Destination.

- **Action:** Triggers the addition of a route record, ultimately passing data to `RouteDAO.addRoute()`.

Flight Management Screen (`FlightPanel.java`)

- **Purpose:** Facilitates Step 2 of the operational flow: "Create Flight" (mirroring "Create Drive").
- **Fields:** Captures Route ID (foreign key linkage) and Departure Date/Time.
- **Action:** Submits flight details to the system, integrating with `FlightDAO.addFlight()`.

Passenger Booking Portal (`BookingPanel.java`)

- **Purpose:** Facilitates the passenger operational flow: "Passenger Books" and "Booking Stored in Database".
- **Fields:** Captures Passenger ID and the target Flight ID.
- **Action:** Submits the booking with a default status of "Pending", mapping to `BookingDAO.bookFlight()`.

Booking Confirmation Screen (`ConfirmationPanel.java`)

- **Purpose:** Allows airline administrators to evaluate and confirm bookings (mirroring the Shortlisting screen).
- **Fields:** Captures the Booking ID and uses a dropdown (`JComboBox`) to select the new status (e.g., "Confirmed", "Cancelled").
- **Action:** Updates the status via `BookingDAO.updateStatus()`.

Seat Allocation Screen (`SeatAllocationPanel.java`)

- **Purpose:** The final step of the reservation workflow (mirroring Interview Scheduling).
- **Fields:** Maps a confirmed Booking ID to a specific Seat Number and Class (e.g., "12A", "Economy").
- **Action:** Saves the seat assignment via the DAO layer.

4. Integration Strategy and Execution Order

To avoid the common mistake of building many disconnected screens before completing a single workflow, the integration will be executed in the following structured order:

1. **Navigation First:** Build the `MainDashboard` to establish simple navigation between major screens.
2. **UI to Service:** Connect the form submission actions directly to mapped service layer methods.

3. **Service to DAO:** Ensure service methods correctly invoke the DAO operations engineered in Phase 3.
4. **User Feedback:** Program clear JOptionPane success messages (e.g., "Route added successfully") and specific error messages. Generic error popups with no useful feedback are strictly avoided.
5. **Database Verification:** Verify that the complete flow (e.g., User clicks "Book Flight" $\rightarrow$ DAO executes INSERT INTO bookings) correctly reflects in the MySQL backend.

5. Input Validation Strategy

A major focus of this phase is data integrity. The application visibly validates data before attempting any database commits:

- **Empty Field Checks:** Forms are scanned for empty fields (e.g., `trim().isEmpty()`). If critical data like Route ID or Passenger ID is missing, the action is blocked, and the user receives a targeted warning message.
- **Data Type Checks:** The UI ensures that numerical ID fields are not submitted with alphabetical characters, catching format exceptions before they hit the DAO layer.

6. Project Folder Structure

Building upon the previous structures, the project directory is expanded to accommodate the clean architectural layers:

```

AirlineReservationSystem/
├── lib/
│   └── mysql-connector-j-9.0.0.jar
├── src/
│   ├── db/
│   │   └── DBConnection.java
│   ├── dao/
│   │   ├── RouteDAO.java
│   │   ├── FlightDAO.java
│   │   └── BookingDAO.java
│   ├── model/
│   │   ├── Route.java
│   │   ├── Flight.java
│   │   └── Booking.java
│   ├── service/      <-- Added in Phase 4
│   │   ├── FlightService.java
│   │   └── BookingService.java
│   └── ui/           <-- Added in Phase 4
│       ├── MainDashboard.java
│       ├── RoutePanel.java
│       └── BookingPanel.java
└── Main.java
└── database.sql
  
```

7. Phase 4 Milestone Review Checklist

The integration and UI development will successfully meet the following phase criteria:

- **Workflow Integrity:** One complete workflow (Add Route -> Create Flight -> Passenger Books) runs seamlessly from the UI down to the DB and back.
- **Layer Separation:** Screen actions are exclusively connected to service logic; direct SQL calls from UI event handlers have been completely avoided.
- **Data Security:** Invalid inputs are properly blocked with visible, descriptive feedback[cite: 5].
- **Usability:** With the inclusion of interactive animations and a unified dashboard, the project is highly usable and ready for the milestone review[cite: 5].